

Replication

Steps to exactly reproduce the results reported in the paper.

Environment

- OS: Windows 11 Home (cross-platform compatible — Linux/macOS should work without changes)
- Python: 3.12.10
- Key dependencies: TensorFlow/Keras, OR-Tools CP-SAT, scikit-learn, scipy, pandas (see [requirements.txt](#))
- Hardware: Standard laptop (Intel CPU); no GPU required — all model inference runs on CPU

Random Seed

No fixed global seed is used. Each run independently samples from the dataset and the solver uses random objectives each iteration. Results are averaged over `--runs` independent trials (default 20) and compared with a Wilcoxon signed-rank test. Trial-to-trial variance is low (see std columns in [results_full/summary.csv](#)).

Steps

1. Clone the repository:

```
git clone https://github.com/RohitM42/dt-csp-fairness-tool.git
cd dt-csp-fairness-tool
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Place the required dataset CSVs in [dataset/](#) and model files in [DNN/](#) (see [manual.pdf](#) for exact filenames).
4. Run the full experiment — all datasets, all methods, baseline shared per dataset:

```
python main.py --dataset all --method all --budget 1000 --runs 20
```

This is the recommended single command. `--dataset all` covers all 8 datasets (6 primary + Law School + Communities & Crime). The baseline is run once per dataset and shared across all three method comparisons. Results are saved incrementally after each run and the summary CSV is updated after each dataset completes, so an interrupted run preserves all completed datasets.

To save to a custom directory (e.g. to preserve a previous run):

```
python main.py --dataset all --method all --budget 1000 --runs 20 --results-dir results_v2
```

To run only the 6 primary datasets (comma-separated list):

```
python main.py --dataset kdd,adult,compas,dutch,credit,german --method all --budget 1000 --runs 20
```

Alternatively, run one dataset at a time:

```
python main.py --dataset kdd --method all --budget 1000 --runs 20
python main.py --dataset adult --method all --budget 1000 --runs 20
python main.py --dataset compas --method all --budget 1000 --runs 20
python main.py --dataset german --method all --budget 1000 --runs 20
python main.py --dataset dutch --method all --budget 1000 --runs 20
python main.py --dataset credit --method all --budget 1000 --runs 20
```

For targeted re-runs of a single method:

```
python main.py --dataset kdd --method dt --budget 1000 --runs 20
```

5. Results are written to `results/` (or `--results-dir` if specified):

- `<results-dir>/<dataset>_baseline.csv` — per-run IDI ratios for random search
- `<results-dir>/<dataset>_dt.csv` — per-run IDI ratios for DT-only
- `<results-dir>/<dataset>_csp.csv` — per-run IDI ratios for CSP-only
- `<results-dir>/<dataset>_hybrid.csv` — per-run IDI ratios for Hybrid
- `<results-dir>/summary.csv` — Wilcoxon p-values, IDI means, and timing per method (updated after each dataset completes)

Group Fairness Characterisation

To reproduce the group-level bias characterisation (DPD and EOD per sensitive feature):

```
python characterise.py
```

Output is printed to the terminal and saved to `characterisation.csv` at the repo root. This is deterministic — no randomness is involved and results are identical on every run.

Expected output file: `characterisation.csv` (18 rows, one per dataset-sensitive feature pair). Key values to verify:

- `dutch`, `age`: DPD=0.4793, EOD=0.6364
- `compas`, `Race`: DPD=0.1278, EOD=0.5000
- `communities_crime`, `Black`: DPD=0.0079, EOD=0.0000
- `german`, `AgeInYears`: DPD=0.4282, EOD=1.0000

IDI Counting — Methodology Note

All methods count **unique discriminatory inputs** only. A discriminatory pair (`a`, `b`) is counted once regardless of how many times input `a` is sampled. This is enforced via a `seen_discriminatory` set in the baseline and DT. CSP uses CP-SAT diversity constraints to avoid revisiting discriminatory regions rather than a deduplication set.

Sensitive Feature Flip — Methodology Note

All methods (baseline, DT, CSP, Hybrid) flip **all sensitive features simultaneously** when creating a comparison pair. For example, on KDD (sensitive: sex, race, age), both sex, race, and age are changed in `sample_b`. This is consistent with the reference implementation in `lab4_solution.py` and tests for intersectional discrimination across all protected attributes at once. The baseline previously flipped only the primary sensitive feature (`sensitive[0]`); this was corrected to match the reference and the three comparison methods.

Expected Results

Full 20-run results (budget=1000) from `results_full/`. All methods use all-sensitive-feature flip. p-values from Wilcoxon signed-rank test (paired, two-sided), Bonferroni-corrected threshold $\alpha \approx 0.008$ (0.05/6 primary comparisons).

Note: Authoritative results are from `results_full/` (all 8 datasets, corrected all-sensitive baseline, 20 runs each). Earlier partial runs in `results/` (5 datasets, single-attribute baseline flip) and `results_v2/` (5 datasets, all-sensitive flip) are retained for reference only and should not be used for comparison with the report.

DT vs Baseline (`results_full/`)

Dataset	Baseline mean $\pm$ std	DT mean $\pm$ std	p-value	Significant
kdd	0.5660 $\pm$ 0.0126	0.6557 $\pm$ 0.0357	1.91e-06	Yes
adult	0.4682 $\pm$ 0.0170	0.5630 $\pm$ 0.0309	1.91e-06	Yes
compas	0.5893 $\pm$ 0.0121	0.6662 $\pm$ 0.0213	8.82e-05	Yes
dutch	0.0220 $\pm$ 0.0043	0.1350 $\pm$ 0.0563	5.72e-06	Yes
credit	0.0783 $\pm$ 0.0108	0.1775 $\pm$ 0.0310	8.84e-05	Yes
german	0.3898 $\pm$ 0.0154	0.3382 $\pm$ 0.0285	1.03e-04	Yes (DT lower)

CSP and Hybrid vs Baseline (`results_full/`)

[†]Law School is ablation-only: DT and baseline both achieve IDI $\approx$ 0.000 on real data rows.

Dataset	Baseline	CSP mean $\pm$ std	Hybrid mean $\pm$ std
kdd	0.5660	0.0395 $\pm$ 0.0066	0.1215 $\pm$ 0.0125
adult	0.4682	0.2180 $\pm$ 0.0145	0.2698 $\pm$ 0.0283
compas	0.5893	0.4398 $\pm$ 0.0182	0.4567 $\pm$ 0.0283
dutch	0.0220	0.0332 $\pm$ 0.0038	0.0761 $\pm$ 0.0327
credit	0.0783	0.1505 $\pm$ 0.0104	0.1587 $\pm$ 0.0220
german	0.3898	0.2318 $\pm$ 0.0150	0.2823 $\pm$ 0.0282
law school†	0.0000	0.0199 $\pm$ 0.0040	0.0179 $\pm$ 0.0049

Key Findings

- **DT outperforms the baseline on 5 of 6 primary datasets**, with statistically significant improvements on all 6 (Wilcoxon, all $p < 0.001$). Improvements range from +13% (COMPAS) to +515% (Dutch). DT achieves this at comparable runtime to the baseline — the DT training overhead is negligible across all primary datasets (range: -0.6% to $+2.4\%$).
- **German is the only dataset where DT underperforms the baseline** (0.338 vs 0.390, -13.3%). This is attributed to three factors: the small dataset size (1,000 rows) causing heavy resampling in the guided phase; a relatively uniform discrimination density that leaves the DT with no concentrated subspace to identify; and a multi-valued sensitive feature (PersonStatusSex, 0–3) creating a mismatch between DT threshold splits and the random flip operation.
- **CSP underperforms the baseline on 4 of 6 primary datasets** (KDD, Adult, COMPAS, German). The root cause is a domain/distribution mismatch: CP-SAT generates inputs uniformly within `[min, max]` feature bounds, while the DNN's discrimination is concentrated on the real data distribution. On KDD (36 features, 285k rows), CSP achieves only 0.040 vs baseline 0.566. CSP exceeds the baseline on Dutch (+51%) and Credit (+92%), where the discriminatory subspace is more accessible to solver-based search.
- **Hybrid underperforms the baseline on 4 of 6 primary datasets** (same pattern as CSP), beating it on Dutch and Credit. Hybrid's Phase 2 uses CSP constrained to DT-identified rule regions — better than unconstrained CSP but still generating synthetic inputs off the real data distribution. DT-only outperforms Hybrid on all six primary datasets because DT Phase 2 continues sampling real data rows, not synthetic ones.
- **DT is the recommended method.** It is black-box (no model internals required), interpretable (extracted decision rules describe which feature combinations drive discrimination), and achieves the highest IDI ratios at no additional computational cost over random search.